How writes keep the copies current, in the write's own transaction

Insert a product p9 (price 6) under store S2

one transaction

1. Save the row: Products (p9, 6, S2)
the application's ordinary save

↓

2. fn_ResourceAncestors(S2), inside the database:
S2, R1, C1, D1, root (the active path, within the depth bound)
the same ancestor rule the access check uses

↓

3. Insert one entry per ancestor into every copy:
Price copy: (S2,6,p9) (R1,6,p9) (C1,6,p9) (D1,6,p9) (root,6,p9)
Id copy: (S2,p9) (R1,p9) (C1,p9) (D1,p9) (root,p9)
... one line per index on Products

↓

Commit, or roll back, together.

Cost: levels × copies entries, here 5 × 2.
Measured: 1.4 ms to 5.7 ms per insert with three copies.

Move region R1 from chain C1 to chain C2

one transaction

1. Save the resource: R1.ParentId = C2
the resource row changes; the product rows do not

↓

2. Walk R1's subtree (a recursive query, bounded by the depth):
R1, S1, S2, ... and every product under them
only moved or deactivated resources are walked

↓

3. In every copy, delete those rows' entries and reinsert them along the new path:
before: (S1,3,p1) (R1,3,p1) (C1,3,p1) (D1,3,p1) (root,3,p1)
after: (S1,3,p1) (R1,3,p1) (C2,3,p1) (D2,3,p1) (root,3,p1)

↓

Commit, or roll back, together.

Cost: subtree rows × levels × copies.
Measured: 501 ms to move 2,000 products with three copies.

What the copies never do

They never decide access. Every page is checked against fn_IsResourceAccessible before it is returned, so a stale or wrong entry can hide a row and can never show one.

Writes that bypass the mapper

Bulk loads, raw SQL and bulk updates do not pass through the save. One command, RebuildPageOrdersAsync, deletes and rewrites every copy from the application tables and the resource graph. The same rebuild fills a new copy after an index is added.